

ZeroThink 1.0: A Reproducible Architecture for Evidence-Gated AI Agent Services

From sovereign-reasoning notes to a falsifiable deployed-systems specification

Shafaet Brady Hussain Independent researcher, United Kingdom 18 August 2026

Abstract

I present ZeroThink as an evidence-gated architecture for deployed AI agent services. Earlier ZeroThink notes used metaphors including sovereign reasoning, Scout/General roles, superposition, and wavefunction collapse. This paper replaces those metaphors with an operational specification: a request enters a policy boundary; evidence-gathering and candidate-generation stages produce typed artifacts; a verifier applies source, security, and output constraints; and an orchestrator either returns a bounded response, requests repair, or abstains. The term quantum-inspired refers only to classical branching and selection. No quantum computation, quantum advantage, consciousness, guaranteed truth, or hallucination elimination is claimed.

The architecture is evaluated using two dated releases. A public-surface study executed 23 predefined checks across six routes, six command-line API contracts, four agent API contracts, and seven static integration checks; all 23 passed in the 9 July 2026 snapshot, and a narrow token-pattern scan reported zero hits. A separate authenticated routing experiment used ten exact-answer tasks. Three cloud lanes scored 10/10, one scored 7/10, and two local OpenZero wrapper lanes scored 3/10; one additional local agent lane and one cloud lane were unavailable. These measurements demonstrate routing and expose wrapper/runtime defects; they do not show that ZeroThink improves the intelligence of underlying models. The release supplies the architecture, state machine, evidence model, threat boundaries, source hashes, scored data, and rejection criteria so that later implementations can be tested rather than accepted by description.

Keywords: AI agents; orchestration; provenance; local-first AI; reproducibility; authentication boundaries; abstention; systems evaluation

1. Introduction

An AI product is more than a model. Its behaviour depends on authentication, routing, prompts, tool permissions, retrieval, state, provider availability, local runtimes, output validation, and the interface through which results reach an operator. A model benchmark can therefore be accurate while the deployed service fails, and a healthy web service can route to a weak or misconfigured model. ZeroThink was created to study and implement this systems layer.

My earlier notes described a Scout that gathers information, a General that decides, five candidate perspectives, a sovereign answer, and simulated quantum logic. Those terms expressed design intent but were too easy to read as stronger scientific claims. In this specification:

- a **Scout** is an evidence-acquisition stage with no authority to approve an answer;
- a **candidate lane** is an ordinary classical model or deterministic procedure;
- a **General** is a policy-constrained orchestrator, not an infallible decision maker;
- **collapse** means ranking, rejecting, or selecting classical candidate outputs;
- **sovereignty** means explicit operator control, inspectable boundaries, and replaceable providers;
- **truth** is not an output type; the system can return only claims with evidence status and limitations.

The contribution is not a new foundation model. It is a testable service architecture that separates evidence acquisition, candidate generation, verification, routing, and release decisions. The paper consolidates two unpublished overlapping Zenodo drafts and supersedes their unqualified claims. The January 2026 ZeroThink Architecture technical note remains a historical record rather than evidence of current validity.

2. Research questions

This paper asks four bounded questions:

1. Can the ZeroThink design vocabulary be expressed as typed components and state transitions rather than metaphors?
2. Can a deployed implementation be evaluated separately at public-boundary, routing, and model-quality layers?
3. Do the released measurements support claims about boundary behaviour and routing without implying model superiority?
4. Which claims must be rejected until broader or independent evidence exists?

3. Architecture

3.1 Components

Let a ZeroThink deployment be the tuple

$ZT = (I, P, E, C, V, O, R, L)$

where I is request intake, P is policy and identity context, E is evidence acquisition, C is candidate generation, V is verification, O is orchestration, R is response release, and L is an append-only event/evidence log. Implementations may omit optional candidate or retrieval lanes, but they must not merge authority in a way that lets untrusted evidence approve itself.

Component	Input	Output	Authority boundary
Intake I	request, declared mode	normalized request	cannot call providers before policy preflight
Policy P	identity, permissions, mode	permitted route set	denies unsupported or unauthenticated protected actions
Evidence E	query plan, permitted sources	evidence records	supplies material; cannot approve a final answer

Component	Input	Output	Authority boundary
Candidates C	request, bounded context	candidate records	may be deterministic, local-model, or provider-model lanes
Verifier V	candidates, evidence, schema	findings and scores	can reject, require repair, or require abstention
Orchestrator O	policy, findings, candidates	release decision	cannot override hard policy or missing-evidence gates
Release R	approved response record	response or abstention	emits declared provider/model and limitations when available
Log L	typed events	trace commitments	records state without storing secrets in public artifacts

3.2 Typed records

Every candidate record should contain at least `candidate_id`, `lane`, `model_or_rule`, `prompt_profile`, `created_at`, `content`, and `content_hash`. Evidence records should contain `source_id`, `source_type`, `retrieved_at`, `locator`, `excerpt_hash`, `licence_or_access_status`, and `claim_links`. Verification findings should contain `check_id`, `severity`, `result`, `evidence_refs`, and `repairability`.

The public trace may contain hashes and redacted metadata. Provider keys, bearer tokens, private prompts, customer data, and raw private retrieval content are outside the public record.

3.3 State machine

The minimum lifecycle is:

```
RECEIVED -> PREFLIGHTED -> EVIDENCE_READY -> CANDIDATES_READY -> VERIFIED -> RELEASED
```

with alternative terminal or repair transitions:

```
PREFLIGHTED -> DENIED EVIDENCE_READY -> ABSTAINED CANDIDATES_READY -> REPAIR -> VERIFIED VERIFIED -> ABSTAINED ANY_NONTERMINAL -> FAILED
```

The transition guard for RELEASED is:

```
permit(P, request) AND schema_valid(response) AND no_hard_failure(V) AND evidence_gate(mode).
```

This guard does not prove factual truth. It proves only that the implementation satisfied its declared release conditions for that trace.

3.4 Candidate diversity

The earlier Pentagon Protocol proposed five perspectives: direct, skeptical, lateral, ethical, and data-oriented. In this specification these are optional prompt or algorithm profiles, not independent realities. A deployment must record whether lanes use distinct models, distinct prompts over one model, deterministic rules, or duplicated calls. Diversity is an experimental variable and must not be inferred from labels alone.

3.5 Selection and abstention

For candidates `c_i`, an implementation may calculate a bounded score

$$S(c_i) = w_e E_i + w_f F_i + w_p P_i + w_s S_i - w_r R_i$$

where E_i is evidence coverage, F_i format/constraint compliance, P_i policy alignment, S_i source quality, and R_i identified risk. Weights, component definitions, and thresholds must be declared for a scored experiment. When they are not declared, the system should report selection as heuristic rather than mathematical optimisation.

Abstention is required when a hard policy check fails, required evidence is absent, all candidates fail the schema, a tool result is stale beyond the allowed window, or a high-risk claim lacks its required authority. The orchestrator may not convert missing evidence into confidence by adding more prose.

4. Threat and failure model

The architecture assumes that model outputs, retrieved documents, webpages, tool responses, and candidate messages can be wrong or adversarial. It does not assume that the orchestrator model is trusted merely because it appears later in the pipeline.

Threat or failure	Required response
Unauthenticated protected request	deny before generation or tool execution
Prompt injection in retrieved content	treat as data; do not grant permissions or new goals
Provider or local runtime unavailable	record unavailable lane; use an allowed fallback or abstain
Candidate violates required schema	repair once under a bounded policy or reject
Missing citation/evidence for gated claim	qualify, retrieve authorised evidence, or abstain
Stale telemetry or evidence	mark stale and refuse freshness-dependent conclusions
Secret-shaped output	withhold output, investigate, and rotate affected credentials if real
Model/wrapper contamination	separate direct-model and wrapped-route evaluation
Unsupported tool or interface	fail closed rather than infer an actuator or command signature

The architecture is not a security boundary by itself. Secure deployment also requires ordinary application security, dependency management, key custody, rate limits, isolation, monitoring, incident response, and independent review.

5. Implementation mapping

The 9 July 2026 public snapshot provides implementation evidence for parts of this model. The public runner observed authentication guards, device authorization, safe local identity/capability branches, and six public routes. A local source snapshot contained declared markers for OpenZero routing, a local model map, direct-mode restrictions, Research Paper Creator actions and validation, and the command-line device protocol.

The source hashes recorded in that dated evaluation were:

File	Bytes	SHA-256
api_agent.php	135,935	fdcdf8b61a020cf93b42c13d32dd91c36e2899bdde7c639722605839ca061dca
api_cli.php	5,444	a161c484d28021091fde67d00ad3be31f5cb58ecd1c628c88dc788980e36f4a0
research-paper-creator.php	73,950	f472979502edbd34f5d5b844ed239846ed68a49c45683cbcf44fadecd685d748

These hashes bind the static observations to local files. They do not prove that the same files were deployed live.

6. Evaluation methods

6.1 Public-boundary protocol

The public-surface protocol used no account cookies, real provider keys, private server credentials, or attempts to bypass authentication. It performed three requests to each of six public pages, exercised six command-line API cases and four agent API cases, inspected seven predeclared local integration markers, and scanned captured responses for selected token-shaped patterns.

6.2 Authenticated routing protocol

The authenticated routing experiment used an authorised ZeroThink account and a temporary bearer token. Provider credentials remained in the server-side account vault and were neither printed nor exported. The token was revoked after the run and cleanup reported no active benchmark tokens. ZeroTrade sources and credentials were excluded.

Ten exact-answer items covered arithmetic, temporal reasoning, quantifier logic, sorting, Python range evaluation, inventory arithmetic, structured JSON, time arithmetic, a density question, and the mislabeled-boxes problem. The scorer used exact expected answers without an LLM judge. This is a diagnostic suite, not a substitute for established broad benchmarks or human evaluation.

6.3 Layer separation

Three layers are reported independently:

1. **Boundary behaviour:** observable public routes and denial/handshake contracts.
2. **Routing behaviour:** whether authenticated requests reached declared provider or local lanes.
3. **Candidate quality:** task accuracy and latency for the selected model through that route.

Passing one layer does not imply passing another.

7. Results

7.1 Public-boundary snapshot

All 23 predefined checks passed in the 9 July 2026 snapshot.

Area	Passed / total	Bounded interpretation
Public pages	6 / 6	selected routes returned expected markers in three requests each
CLI API	6 / 6	tested input, authentication, device-start, and pending contracts matched
Agent API	4 / 4	tested unauthenticated requests were blocked; two local branches matched
Static snapshot	7 / 7	selected markers existed in three hashed files
Token-pattern scan	0 hits	no selected token-shaped pattern appeared in captured responses

The mean latency across benchmark rows with latency was 0.134 seconds, with a median of 0.061 seconds. These are descriptive observations, not availability or service-level measurements.

7.2 Authenticated routing snapshot

Lane	Selected model	Correct / 10	Mean latency (s)	Status
Groq account route	openai/gpt-oss-120b	10	0.850	available
OpenAI account route	gpt-5.4	10	1.204	available
Gemini account route	gemini-3.5-flash	10	3.272	available
NVIDIA account route	meta/llama-3.3-70b-instruct	7	29.482	available
OpenZero GLM4 wrapper route	glm4:9b-q5	3	8.494	available
OpenZero Spectra8 wrapper route	spectra8-q8	3	75.703	available
xAI account route	none	not scored	not applicable	key rejected
OpenZero agent route	none	not scored	not applicable	local runtime timed out

The three 10/10 results show that ZeroThink routed the tasks to strong provider models. They do not show that ZeroThink improved those models. Conversely, the local wrapper results reveal a deployment problem: the same Spectra8 family had scored 9/10 in a separate direct OpenZero snapshot, but only 3/10 through this wrapper route. The recorded symptoms included long timeouts and probe/context contamination. This difference is evidence for evaluating wrappers independently from model artifacts.

8. Claim ledger

Claim	Status	Evidence or rejection rule
ZeroThink can enforce selected public authentication and device-flow contracts	supported for the dated 23-check snapshot	public result matrix and machine-readable summary
ZeroThink can route an authorised account to multiple provider/local lanes	supported for the dated routing snapshot	scored rows and declared selected models
ZeroThink improves foundation-model intelligence	rejected	experiment did not isolate or demonstrate improvement
ZeroThink eliminates hallucinations	rejected	no architecture can infer this from route or exact-answer checks
ZeroThink performs quantum computation	rejected	all reported branching and selection were classical
Five labelled lanes constitute five independent models or realities	not established	implementation must declare model and prompt identity per lane
ZeroThink guarantees truth or a sovereign answer	rejected	release gates produce bounded decisions, not epistemic certainty
The system is secure	not established	small self-evaluation is not an audit or certification
The architecture is reproducible	partially supported	public artifacts reproduce the dated checks; full deployment is not released

9. Conformance profile

A system may call itself **ZeroThink Core conformant** only if it:

1. implements explicit request preflight before protected generation or tool use;

2. represents evidence, candidates, findings, and release decisions as distinguishable records;
3. prevents evidence or candidate content from granting permissions;
4. has a declared abstention path;
5. logs provider/model or deterministic rule identity for evaluated lanes;
6. separates live observations from static-source observations;
7. publishes dated evaluation parameters and negative results;
8. does not describe classical branching as quantum computation.

ZeroThink Evidence conformant additionally requires claim-to-source links, evidence freshness status, hash commitments, and explicit missing-evidence findings. **ZeroThink Reproducible conformant** additionally requires executable tests, machine-readable results, dependency/runtime instructions, immutable release hashes, and a limitations statement.

Conformance is self-declared unless an independent evaluator repeats it. It is not a security certification.

Executable conformance companion

The release companion under `artifacts/zerothink-architecture-1.0/` turns a narrow subset of this specification into executable checks. A JSON Schema fixes the trace vocabulary and required fields. A dependency-free Python verifier then checks state ordering, release authority, hard-failure denial, evidence-gate completion, and the rule that untrusted content cannot grant permissions. Included positive and negative vectors make the expected decisions inspectable. Passing these tests establishes only conformance to those encoded invariants; it does not prove factual correctness, security, or full implementation of the architecture.

10. Falsifiable evaluation programme

The next evaluation should compare four conditions on the same frozen tasks:

- B0: one-shot base model;
- B1: same model with a structured prompt but no evidence stage;
- B2: ZeroThink evidence and verification stages with one candidate;
- B3: ZeroThink with multiple declared candidate profiles.

Primary outcomes should include exact correctness where possible, unsupported-claim rate, citation precision/recall, schema compliance, abstention appropriateness, latency, cost, and repair count. Candidate diversity should be measured from outputs and model/prompt identities rather than assumed from lane names. The experiment should preregister thresholds, freeze prompts, publish failures, and separate direct-model from wrapped-service runs.

A claim that ZeroThink adds value would be falsified if B2 and B3 do not improve predefined evidence or error outcomes relative to B0/B1, or if any improvement disappears when latency, provider strength, and prompt-token budget are controlled.

11. Limitations

This is an author-led consolidation and self-evaluation, not peer review or independent validation. The public-boundary experiment is small and dated. Three requests per route do not establish uptime, capacity, or geographic performance. Pattern matching cannot prove the absence of secrets. Static markers do not prove live deployment identity or runtime behaviour. The ten exact-answer items are diagnostic and too

small for broad claims. Provider models, keys, and service versions can change. The experiments do not evaluate tool-use safety, prompt-injection resistance, privacy-law compliance, accessibility, human usefulness, long-horizon autonomy, or adversarial security.

The architecture is deliberately descriptive at some points: it permits different verifier and scoring implementations. No theorem shows that the proposed score produces correct answers. The use of the word sovereign describes control and auditability, not political, legal, conscious, or epistemic sovereignty.

12. Conclusion

ZeroThink is best understood as an evidence-gated service architecture, not as a claim that an orchestration wrapper creates a new form of model intelligence. Its useful scientific subject is the boundary between request policy, evidence, candidate generation, verification, routing, abstention, and release. The dated evaluations show two complementary facts: selected public contracts behaved as expected in a 23-check snapshot, and authenticated routing exposed both capable cloud lanes and serious local-wrapper degradation. Publishing those successes and failures makes the architecture falsifiable.

The consolidated specification supersedes the strongest unqualified language in the earlier drafts. Future claims should be earned through frozen baselines, declared lane identities, machine-readable traces, negative results, and independent reruns.

Data and code availability

The public research repository is <https://github.com/ResearchForumOnline/research>. Dated public-boundary and authenticated-routing CSV/JSON artifacts are stored under `data/benchmarks/`. The historical ZeroThink record is <https://doi.org/10.5281/zenodo.18305187>. The two overlapping unpublished Zenodo uploads should be treated as source drafts, not separate validated publications.

Ethics, privacy, and security

The public-boundary study avoided account cookies, real provider credentials, private server credentials, customer data, and unauthorised access. The authenticated study used an authorised account, retained provider keys server-side, redacted account identifiers, revoked temporary benchmark tokens, and excluded ZeroTrade. Public packages must exclude private prompts, keys, raw customer or DNA data, locked evaluations, and exploit-enabling operational details.

Conflict of interest

I created and maintain components in the TalkToAI, ZeroThink, and OpenZero ecosystem. This is a self-evaluation. I therefore publish negative results and avoid describing the work as an independent audit, certification, or comparative proof of superiority.

AI-use disclosure

AI-assisted tools were used for corpus search, editorial consolidation, code support, and language refinement. I am responsible for the architecture, evidence selection, released artifacts, limitations, and final claims. Generated prose is not treated as an experimental observation.

References

1. Hussain, S. B. *Boundary-Oriented Evaluation of a Deployed AI Agent Service: A Reproducible Public-Surface Case Study*. Zenodo, 2026. <https://doi.org/10.5281/zenodo.21581974>
2. Hussain, S. B. *ZeroThink Architecture*. Zenodo, 2026. <https://doi.org/10.5281/zenodo.18305187>

3. National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1, 2023. <https://doi.org/10.6028/NIST.AI.100-1>
4. National Institute of Standards and Technology. *Technical Guide to Information Security Testing and Assessment*. NIST SP 800-115, 2008. <https://doi.org/10.6028/NIST.SP.800-115>
5. OWASP Foundation. *Application Security Verification Standard 5.0.0*. 2025. <https://owasp.org/www-project-application-security-verification-standard/>
6. Mitchell, M., et al. Model Cards for Model Reporting. *FAT* 2019. <https://doi.org/10.1145/3287560.3287596>
7. Sculley, D., et al. Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems* 28, 2015.
8. World Wide Web Consortium. *PROV-O: The PROV Ontology*. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-o/>

Appendix A. Released result files

Public-boundary results are in data/benchmarks/zerothink-system-benchmark-2026-07-09.csv and its -summary-2026-07-09.json companion. Authenticated-routing results are in data/benchmarks/zerothink-authenticated-intelligence-benchmark-2026-07-09.csv and its summary JSON; the separate Spectra8 direct-route snapshot uses the same stem with -spectra8-only before the extension.

Appendix B. Supersession statement

This paper consolidates the unpublished uploads titled *ZeroThink Swarm Intelligence Architecture* and *ZeroThink: The Sovereign Reasoning Layer*. It preserves their design vocabulary as historical context but supersedes claims that the system guarantees truth, prevents hallucination, performs quantum computation, holds contradictory realities in a physical sense, or has been independently validated merely because the work was discussed, routed, or viewed by an institution or company.